Express.js Web Application

In this section, you will learn how to create a web application using Express.js.

Express.js provides an easy way to create web server and render HTML pages for different HTTP requests by configuring routes for your application.

Web Server

First of all, import the Express.js module and create the web server as shown below.

app.js: Express.js Web Server — Copy

```
var express = require('express');
var app = express();

// define routes here..

var server = app.listen(5000, function () {
    console.log('Node server is running..');
});
```

In the above example, we imported Express.js module using require() function. The express module returns a function. This function returns an object which can be used to configure Express application (app in the above example).

The app object includes methods for routing HTTP requests, configuring middleware, rendering HTML views and registering a template engine.

The app.listen() function creates the Node.js web server at the specified host and port. It is identical to Node's http.Server.listen() method.

Run the above example using `node app.js` command and point your browser to *http://localhost:5000*. It will display **Cannot GET /** because we have not configured any routes yet.

Configure Routes

Use app object to define different routes of your application. The app object includes get(), post(), put() and delete() methods to define routes for HTTP GET, POST, PUT and DELETE requests respectively.

The following example demonstrates configuring routes for HTTP requests.

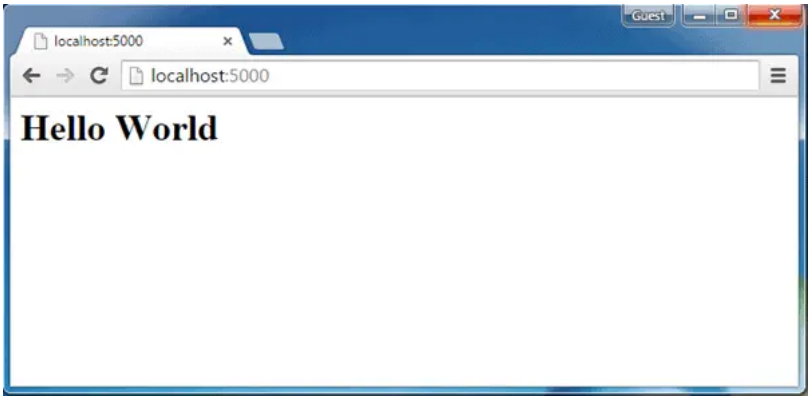
Example: Configure Routes in Express.js

Copy

```
var express = require('express');
var app = express();

app.get('/', function (req, res) {
    res.send('<html><body><h1>Hello World</h1></body></html>');
});

app.post('/submit-data', function (req, res) {
    res.send('POST Request');
});

app.put('/update-data', function (req, res) {
    res.send('PUT Request');
});

app.delete('/delete-data', function (req, res) {
    res.send('DELETE Request');
});

var server = app.listen(5000, function () {
    console.log('Node server is running..');
});
```

In the above example, app.get(), app.post(), app.put() and app.delete() methods define routes for HTTP GET, POST, PUT, DELETE respectively. The first parameter is a path of a route which will start after base URL. The callback function includes request and response object which will be executed on each request.

Run the above example using `node server.js` command, and point your browser to *http://localhost:5000* and you will see the following result.



Express.js Web Application

Handle POST Request

Here, you will learn how to handle HTTP POST request and get data from the submitted form.

First, create Index.html file in the root folder of your application and write the following HTML code in it.

Example: Configure Routes in Express.js

Copy

```
<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head>
    <meta charset="utf-8" />
    <title></title>
</head>
<body>
    <form action="/submit-student-data" method="post">
        First Name: <input name="firstName" type="text" /> <br />
        Last Name: <input name="lastName" type="text" /> <br />
        <input type="submit" />
    </form>
</body>
</html>
```

Body Parser

To handle HTTP POST request in Express.js version 4 and above, you need to install middleware module called body-parser. The middleware was a part of Express.js earlier but now you have to install it separately.

This body-parser module parses the JSON, buffer, string and url encoded data submitted using HTTP POST request. Install body-parser using NPM as shown below.

```
npm install body-parser --save
```

Now, import body-parser and get the POST request data as shown below.

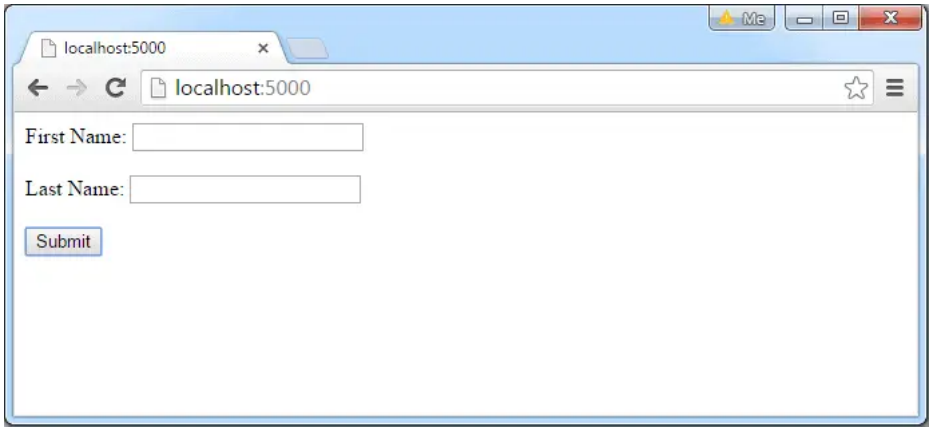
app.js: Handle POST Route in Express.js

Copy

```
var express = require('express');
var app = express();

var bodyParser = require("body-parser");
app.use(bodyParser.urlencoded({ extended: false }));

app.get('/', function (req, res) {
    res.sendFile('index.html');
});

app.post('/submit-student-data', function (req, res) {
    var name = req.body.firstName + ' ' + req.body.lastName;

    res.send(name + ' Submitted Successfully!');
});

var server = app.listen(5000, function () {
    console.log('Node server is running..');
});
```

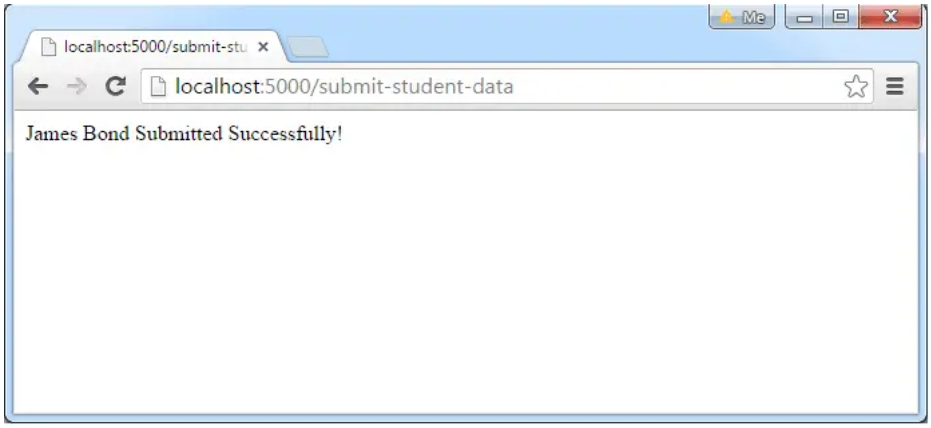
In the above example, POST data can be accessed using req.body. The req.body is an object that includes properties for each submitted form. Index.html contains firstName and lastName input types, so you can access it using req.body.firstName and req.body.lastName.

Now, run the above example using `node server.js` command, point your browser to *http:///localhost:5000* and see the following result.



HTML Form to submit POST request

Fill the First Name and Last Name in the above example and click on **submit**. For example, enter "James" in First Name textbox and "Bond" in Last Name textbox and click the submit button. The following result is displayed.



Response from POST request

This is how you can handle HTTP requests using Express.js.

Want to check how much you know Node.js?

Start Node.js Test

Share Tweet Share Whatsapp

‹ Previous Next ›

.NET Tutorials

- C#
- Object Oriented C#
- ASP.NET Core
- ASP.NET MVC
- LINQ
- Inversion of Control
- Web API